```
"""
nexus.core.interfaces
=====================

Abstract contracts
(``typing.Protocol``) that every
concrete implementation
in later modules
(Claude/GPT/Gemini/Ollama adapters,
Qdrant/Weaviate memory,
Whisper/Deepgram STT, ElevenLabs/XTTS
TTS, ADB/Appium/Playwright executors)
must satisfy.

Why ``Protocol`` instead of
``abc.ABC``:
    Structural typing lets third-
party or plugin-provided
implementations
    satisfy an interface without
importing NEXUS's base classes or
joining
    an inheritance hierarchy. This
matters for the plugin system (later
    module): a plugin author can
write a plain class with the right
```

```
method
    signatures and it will type-check
as a valid provider, no NEXUS import
    required beyond the Protocol
itself for static analysis.

Every interface here is intentionally
``async def`` — NEXUS is async-first
throughout the agent/voice/automation
layers so that, e.g., a slow ADB call
never blocks the voice pipeline or an
unrelated in-flight LLM request.
"""

from __future__ import annotations

from dataclasses import dataclass,
field
from enum import Enum
from typing import Any, Protocol,
runtime_checkable


# ----------------------------------
----------------------------------
---
# Shared value objects
# ----------------------------------
----------------------------------
```

```
---

class Role(str, Enum):
    """Speaker role in a conversation
turn."""

    SYSTEM = "system"
    USER = "user"
    ASSISTANT = "assistant"
    TOOL = "tool"


@dataclass(frozen=True, slots=True)
class Message:
    """A single conversational turn,
provider-agnostic."""

    role: Role
    content: str
    metadata: dict[str, Any] =
field(default_factory=dict)


@dataclass(frozen=True, slots=True)
class LLMResponse:
    """Normalized response shape
returned by every ``LLMProvider``
implementation."""
```

```
    content: str
    model_name: str
    input_tokens: int
    output_tokens: int
    raw: Any = None  # provider-
specific raw response, kept for
debugging only


@dataclass(frozen=True, slots=True)
class ActionPlan:
    """
    A structured, provider-agnostic
action produced by the Planner Agent.

    This is the ONLY thing an LLM is
ever allowed to produce when it wants
    something executed on a device or
in the browser — never a raw shell
    command or script. See
``nexus.core.interfaces.ActionExecuto
r`` and the
    guardrail engine below for how
this gets validated before it runs.
    """

    action_type: str
    target: str | None  # package
name, URL, file path, window title,
```

```
etc.
    params: dict[str, Any] =
field(default_factory=dict)
    reasoning: str = ""
    requested_by_agent: str = ""


class GuardrailDecision(str, Enum):
    """Outcome of passing an
``ActionPlan`` through the guardrail
engine."""

    ALLOWED = "allowed"
    BLOCKED = "blocked"
    REQUIRES_CONFIRMATION =
"requires_confirmation"


@dataclass(frozen=True, slots=True)
class GuardrailResult:
    """Result of a guardrail check,
including the reason for
auditability."""

    decision: GuardrailDecision
    reason: str
    plan: ActionPlan
```

```
@dataclass(frozen=True, slots=True)
class ExecutionResult:
    """Result of actually running an
approved ``ActionPlan``."""

    success: bool
    output: str
    plan: ActionPlan
    error: str | None = None


@dataclass(frozen=True, slots=True)
class MemoryRecord:
    """A single retrievable unit of
memory (episodic or semantic)."""

    id: str
    content: str
    embedding: list[float] | None
    metadata: dict[str, Any] =
field(default_factory=dict)
    score: float | None = None  #
similarity score, populated on
retrieval only


# ----------------------------------
----------------------------------
---
```

```
# Provider interfaces
# ----------------------------------
----------------------------------
---

@runtime_checkable
class LLMProvider(Protocol):
    """
    Implemented by each
interchangeable LLM backend (Claude,
GPT, Gemini,
    local Ollama models). The rest of
NEXUS depends only on this Protocol,
    never on a specific vendor SDK,
which is what makes model swapping a
    configuration change rather than
a code change.
    """

    async def complete(
        self,
        messages: list[Message],
        *,
        system_prompt: str | None =
None,
        max_tokens: int = 2048,
        temperature: float = 0.4,
    ) -> LLMResponse:
        """Return a single completion
```

```
for the given conversation
history."""
        ...

    async def stream(
        self,
        messages: list[Message],
        *,
        system_prompt: str | None =
None,
        max_tokens: int = 2048,
        temperature: float = 0.4,
    ):
        """
        Yield response text
incrementally as an async generator
of ``str``
        chunks. Used to start TTS
playback before the full response is
        generated, minimizing
perceived latency.
        """
        ...


@runtime_checkable
class SpeechToTextProvider(Protocol):
    """Implemented by each
interchangeable STT backend (Whisper,
```

```
Deepgram, ...)."""

    async def transcribe(self,
audio_bytes: bytes, *, language: str
| None = None) -> str:
        """Transcribe a complete
audio buffer to text."""
        ...

    async def transcribe_stream(self,
audio_chunks):
        """
        Consume an async iterator of
raw PCM audio chunks and yield
partial
        then final transcripts as
they become available.
        """
        ...


@runtime_checkable
class TextToSpeechProvider(Protocol):
    """Implemented by each
interchangeable TTS backend
(ElevenLabs, XTTS-v2, ...)."""

    async def synthesize(self, text:
str, *, voice_id: str, language: str
```

```
= "en") -> bytes:
        """Return synthesized audio
bytes for the given text."""
        ...

    async def synthesize_stream(self,
text: str, *, voice_id: str,
language: str = "en"):
        """Yield audio chunks as they
are generated, for low-latency
playback."""
        ...


@runtime_checkable
class MemoryStore(Protocol):
    """
    Implemented by each
interchangeable memory backend. A
single
    implementation may internally
combine a vector DB (semantic search)
    with a relational store
(structured facts) — callers only see
this
    Protocol.
    """

    async def add(self, record:
```

```
MemoryRecord) -> None:
        """Persist a memory
record."""
        ...

    async def search(
        self, query: str, *, top_k:
int = 5, filters: dict[str, Any] |
None = None
    ) -> list[MemoryRecord]:
        """Return the top-k most
relevant records for a query."""
        ...

    async def delete(self, record_id:
str) -> None:
        """Remove a memory record by
id (e.g. on explicit user request to
forget something)."""
        ...


@runtime_checkable
class ActionExecutor(Protocol):
    """
    Implemented by each automation
backend (ADB, Appium, desktop
    input/window management,
Playwright). Every implementation
```

```
receives
    only an already-guardrail-
approved ``ActionPlan`` — executors
    themselves perform no policy
decisions, only mechanical execution.
    """

    async def execute(self, plan:
ActionPlan) -> ExecutionResult:
        """Execute an approved action
plan and report the outcome."""
        ...

    def supports(self, action_type:
str) -> bool:
        """Whether this executor
knows how to handle the given action
type."""
        ...


@runtime_checkable
class GuardrailEngine(Protocol):
    """
    Implemented by the deterministic
safety layer (see the security
module,
    introduced later). Distinct from
``LLMProvider`` on purpose: guardrail
```

```
    decisions must never be delegated
to a model's own judgment.
    """

    def evaluate(self, plan:
ActionPlan) -> GuardrailResult:
        """Deterministically evaluate
a proposed action plan."""
        ...


@runtime_checkable
class Agent(Protocol):
    """
    Implemented by each LangGraph
node/agent (Conversation, Planner,
    Android, Desktop, Browser,
Coding, Search, Vision, Memory). Kept
    intentionally minimal — the
actual LangGraph state schema is
defined
    where the graph is assembled, not
here, so this core layer doesn't need
    a LangGraph dependency at all.
    """

    name: str

    async def run(self, state:
```

```
dict[str, Any]) -> dict[str, Any]:
        """Process the current graph
state and return updates to merge
into it."""
        ...
```